Rwanda Coding Academy

Nyabihu District
rca.ac.rw
info@rca.ac.rw
0788-548-000

COURSE: ML with Python
CAT 1: Y3, TERM 2
PERIOD: 2hours
Examiner: Louis MUKAMA

Date: 20/02/2024
MARKS: 50 Marks

Instructions

1. Attempt all Questions
2. Use this question paper as your answer-sheet
3. Make your handwriting clear
4. Write your respective names and class on this answer-sheet

Marking guide.

Student details

NAME: CLASS:

CamScanner

Rwanda Coding Academy

Nyabihu District
rca.ac.rw
info@rca.ac.rw
0788-548-000

COURSE: ML with Python
CAT 1: Y3, TERM 2 **Date:** 20/02/2024
PERIOD: 2hours **MARKS:** 50 Marks
Examiner: Louis MUKAMA

Instructions

1. Attempt all Questions
2. Use this question paper as your answer-sheet
3. Make your handwriting clear
4. Write your respective names and class on this answer-sheet

Marking guide.

Student details

Names | Class

CamScanner

Section A

1. Insert the correct syntax for removing rows with
df.dropna() **(1mark)**

2. Insert the correct syntax for replacing empty cells with the value "120".
df.fillna(120) **(1mark)**

3. Insert the correct argument to make sure that the changes are done for the original DataFrame instead of returning a new one.
df.dropna(inplace = True) **(2marks)**

4. What is a correct syntax for changing the 'Firstname' column in the first row to 'John'? **(1mark)**

A. df.loc[0, 'Firstname'] = 'John'
B. df.loc[0]['Firstname'] = 'John'
C. df.loc[0, 'Firstname', 'John']

5. What will happen in the result when the value for 'Duration' is higher than 120? **(1mark)**

```
for x in df.index:
  if df.loc[x, 'Duration'] > 120:
    df.drop(x, inplace = True)
```

A. The Duration value will be set to 120
B. The Duration column will be deleted
C. The entire row will be deleted

6. Insert the correct syntax for removing duplicates in a DataFrame.
df.drop_duplicates() **(1mark)**

7. True or false: A correlation of 0.9 is considered a good correlation. **(1marK)**

8. Insert the correct syntax for specifying that the plot should be of type 'scatter'.
df.plot(kind="scatter", x = 'Duration', y = 'Calories') **(2marks)**

CamScanner

9. Fill in the missing values A, B, and C in the following code snippet to ensure it runs correctly**(3marks)**

sklearn model_selection

```
from [ A ] [ B ] import train_test_split
x_train, x_test, y_train, y_test = [ C ](
    input_data,
    output_data ,
    test_size = 0.2,
    random_state = 42
)
```

train_test_split

10. Examine below code snippet to answer related questions. **(3marks)**

```
#Assume df is a dataframe and initialized before
df["price"] = pd.to_numeric(df["price"], errors="")
```

Select the value(s) that error parameter can take

A. raise
B. coerce
C. ignore
D. downcast
E. jump

Section B

11. Analyse below given dataset and answer related questions.

Manufacturer	Year	Body Type	Transmission	Fuel Type	Seats	Status	Owner Profession	Owner Age
Toyota	2020	Sedan	Automatic	Gasoline	5	Used	Teacher	35
Honda	2019	SUV	Manual	Gasoline	7	New	Engineer	40
Ford	2021	Truck	Automatic	Diesel	2	Used	Business Owner	50
Tesla	2022	Sedan	Automatic	Electric	5	New	IT Specialist	30
Chevrolet	2018	SUV	Automatic	Gasoline	7	Used	Nurse	28
BMW	2021	Coupe	Automatic	Gasoline	4	New	Lawyer	45
Mercedes	2020	Sedan	Automatic	Diesel	5	Used	Lawyer	48
Hyundai	2019	Truck	Manual	Gasoline	2	Used	Marketing Manager	22
...	...	...	...	...	...	...	...	...
Subaru	2022	SUV	Automatic	Gasoline	7	New	Student	34
Nissan	2021	Sedan	Automatic	Gasoline	5	Used	Scientist	705
Hyundai	2022	Sedan	Automatic	Electric	5	Used	Retired	22
Kia	2020	SUV	Automatic	Gasoline	7	New	Artist	29
Volkswagen	2021	Coupe	Manual	Hybrid	4	New	Developer	31
Mazda	2022	Convertible	Automatic	Gasoline	3	Used	Photographer	27
Tesla	2020	Convertible	Automatic	Diesel	3	New	Chef	44
Tesla	2022	Sedan	Automatic	Diesel	5	New	Chef	44

a) If you want to find the standard deviation of owner_age for cars manufactured after the year 2000, what code would you use? **(2marks)**

b) Provide a code snippet that adds car_age feature in existing dataset**(2marks)**

a) std_dev_owner_age = df[df["year"] > 2000]["owner_age"].std()
std_dev_owner_age.

CamScanner

b) from datetime import datetime
current_year = datetime.now().year # getting current year
df["car_age"] = current_year - df["year"] # calculating car_age.
df[["manufacturer", "year", "car_age"]] # displaying relevant columns

12. Analyze the following refrigerator sales dataset to extract meaningful insights and patterns related to pricing, sales trends, and consumer demographics. The dataset contains attributes such as manufacturer, energy rating, capacity, status, owner profession, and selling price.

Manufacturer	Year	Energy Rating	Capacity	Status	Profession	Owner Age	Price (USD)	Selling Date
Samsung	2020	A+	300L	Used	Teacher	35	800	2021-06-15
LG	2019	A++	350L	New	Engineer	40	900	2022-03-10
Panasonic	2021	A+++	400L	Used	Artist	50	1,200	2023-01-20
Sharp	2022	A	250L	New	IT Specialist	30	700	2022-09-05
Samsung	2018	A+	280L	Used	Nurse	28	600	2021-11-11
LG	2021	A++	320L	New	Lawyer	45	1,000	2022-05-25
Panasonic	2020	A	310L	Used	Lawyer	48	1,100	2023-02-15
...	...	...	...	...	...	...	...	...
Sharp	2022	A+++	450L	New	Student	34	950	2022-07-30
Samsung	2021	A++	275L	Used	Scientist	40	750	2021-12-01
LG	2022	A+	360L	New	Retired	62	850	2022-08-14
Panasonic	2020	A	290L	New	Artist	29	620	2023-03-03
Sharp	2021	A+++	310L	New	Developer	31	1,050	2022-10-10
Samsung	2022	A++	240L	Used	Photographer	27	700	2022-11-20
LG	2021	A+	260L	Used	Chef	44	680	2022-02-28

a) Apply a binary encoding technique to convert the "Status" column into a numerical format. **(2.5marks)**

b) Create a new column 'season' in existing dataset classifying each refrigerator's selling date into its respective season, (Winter[12, 1, 2], Spring[3, 4, 5], Summer[6, 7, 8], and Fall) **(2.5marks)**

c) Determine the average selling price for each season. **(2.5marks)**

d) Evaluate the correlation between the owner age and the refrigerator price. **(2.5marks)**

13. Refer to below given hypothetical dataset, that reflects the relationship between house sizes and their prices, to answer ALL related questions:

Size (m^2)	Price (rwfs)
800	150,000
1,200	200,000
1,500	250,000
1,800	300,000
2,000	350,000

For a simple linear regression given by model $Y = \beta_0 + \beta_1 X$ where β_0 is an intercept calculated by $\bar{y} - \beta_1 \bar{x}$, and β_1 being the slope calculated by $\frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$. The $\bar{y}$ and $\bar{x}$ are means of dependent and independent variables respectively.
Calculate the coefficient of determination (R^2) and Root Mean Squared Error (RMSE) metrics that evaluates the model **(20marks)**

End

CamScanner

(12) a) df["status"] = df["Status"].map({"New": 1, "Used": 0})

b) Extract month and season

df['Month'] = df['selling Date'].dt.month # give 0.5 marks

df['season'] = df['Month'].apply(lambda x: 'winter' if x in [12, 1, 2] else
'spring' if x in [3, 4, 5] else
'Summer' if x in [6, 7, 8] else
'Fall') # give 2 marks

c) Simple seasonal analysis : df.groupby('Season')['Price (USD)'].mean()

d) Correlation = df["Owner Age"].corr(df["Price (USD)"])

(13) Some calculations:

Mean of X (House Sizes) $\bar{x} = \frac{800 + 1200 + 1500 + 1800 + 2000}{5} = 1460$

Mean of Y (House Prices) $\bar{y} = \frac{150000 + 200000 + 250000 + 300000 + 350000}{5} = 250000$

Size (m^2)	Price (rwfs)	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
800	150 000	-660	-100000	66 000000	435 6000
1200	200000	-260	-50000	13 000 000	67 600
1500	250 000	40	0	0	1600
1800	300000	340	50000	17 000000	115 600
2000	350000	540	100000	54 000000	291600
				$\Sigma(x_i-\bar{x})(y_i-\bar{y}) =$ 150000 000	$\Sigma(x_i-\bar{x})^2 =$ 912 000

slope $\beta_1 = \frac{\Sigma(x_i - \bar{x})(y_i - \bar{y})}{\Sigma(x_i - \bar{x})^2} = \frac{150000000}{912000} = 164.47$

Intercept $\beta_0 = \bar{y} - \beta_1 \bar{x}$

$= 250000 - (164.47 \times 1460)$

$= 250000 - 240126.2$

$= 9873.8$

Thus the Regression equation is $y = 9873.8 + 164.47x$

x_i size (m^2)	y_i Actual Price (rwfc)	Predicted $\hat{y}_i$	$(y_i - \hat{y}_i)^2$	$(y_i - \bar{y})^2$
800	150 000	141449.8	73105920	10 000000000
1200	200000	207237.8	5238547	2500 000 000
1500	250 000	256578.8	43280609	0
1800	300000	305919.8	35044032	2500 000 000
2000	350000	338813.8	125131070	10 000 000000
			328947381	25 000 000000

$$R^2 = 1 - \frac{\Sigma(y_i - \hat{y}_i)^2}{\Sigma(y_i - \bar{y})^2}$$

$$= 1 - \frac{328947381}{25000000000}$$

$$= 0.98684210476$$

$$RMSE = \sqrt{\frac{1}{n} \Sigma (y_i - \hat{y}_i)^2}$$

$$= \sqrt{\frac{328947381}{5}}$$

$$= 8,111.071211621804$$

CamScanner